

```

import tkinter as tk
from tkinter import *
from tkinter import ttk, messagebox
import mysql.connector as mys
from turtle import Screen, Turtle
import random
from decimal import Decimal
import os

# ----- DATABASE CONFIG -----
# Store your MySQL password as an environment variable instead of hardcoding it:
#   Windows (cmd): set DB_PASSWORD=your_password_here
#   Windows (PowerShell): $env:DB_PASSWORD="your_password_here"
#   macOS/Linux: export DB_PASSWORD=your_password_here
DB_HOST = "localhost"
DB_USER = "root"
DB_PASSWORD = os.environ.get("DB_PASSWORD", "")
DB_NAME = "adis2"

def get_connection(with_db=True):
    if with_db:
        return mys.connect(host=DB_HOST, user=DB_USER, passwd=DB_PASSWORD, databa
        return mys.connect(host=DB_HOST, user=DB_USER, passwd=DB_PASSWORD)

# ----- DATABASE CREATION -----
def init_db():
    myconn = get_connection(with_db=False)
    mycur = myconn.cursor()
    mycur.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    mycur.execute(f"USE {DB_NAME}")
    # Bank table
    mycur.execute("""
        CREATE TABLE IF NOT EXISTS Bank(
            ACCTNO INT PRIMARY KEY,
            NAME VARCHAR(100),
            EID VARCHAR(50),
            PHO BIGINT,
            BAL DECIMAL(10,2)
        )
    """)
    # Loan table
    mycur.execute("""
        CREATE TABLE IF NOT EXISTS Loan(
            ACCTNO INT PRIMARY KEY,

```

```

        PRINCIPAL DECIMAL(10,2),
        INTEREST_RATE DECIMAL(5,2),
        PERIOD INT,
        TOTAL_PAYABLE DECIMAL(10,2)
    )
    """
    myconn.commit()
    myconn.close()

# ----- TURTLE ANIMATION -----
def rainbow_turtle():
    s = Screen()
    s.bgcolor("black")
    t = Turtle()
    t.speed(0)
    colors = ["red", "orange", "yellow", "green", "blue", "indigo", "violet"]

    t.penup()
    t.goto(0, 150)
    t.pendown()

    x = 0
    y = 0
    for i in range(125):
        t.pencolor(random.choice(colors))
        t.forward(x)
        t.right(y)
        x += 2
        y += 1

    t.penup()
    t.goto(0, 0)
    t.pendown()
    t.pencolor("white")
    t.write("Welcome to Cullen Bank!!", align="center", font=("Arial", 24, "bold")
    t.hideturtle()

    def on_click(x, y):
        s.bye()
        login_screen()

    s.onclick(on_click)
    s.mainloop()

# ----- LOGIN -----
def login_screen():

```

```

def funclear():
    txtUser.delete(0, END)
    txtpass.delete(0, END)

def funlogin():
    username = txtUser.get()
    password = txtpass.get()
    if username == "admin" and password == "adis2":
        messagebox.showinfo("Login", "Login Successful")
        root.destroy()
        main_menu()
    else:
        messagebox.showerror("Login", "Invalid Credentials")

root = Tk()
root.geometry("350x200")
root.title("Login")
root.config(bg="#B0E0E6")
Label(root, text="Username:", bg="#B0E0E6").place(x=30, y=30)
txtUser = Entry(root)
txtUser.place(x=120, y=30)
Label(root, text="Password:", bg="#B0E0E6").place(x=30, y=70)
txtpass = Entry(root, show="*")
txtpass.place(x=120, y=70)
Button(root, text="Login", command=funlogin, bg="#4682B4", fg="white").place(
Button(root, text="Clear", command=funclear, bg="#4682B4", fg="white").place(
root.mainloop()

# ----- BUTTON HOVER EFFECT -----
def on_enter(e):
    e.widget['background'] = '#5A9BD5'

def on_leave(e):
    e.widget['background'] = '#4682B4'

# ----- CRUD & BANK OPERATIONS -----
def insert_record():
    def clear():
        txtACCT.delete(0, END); txtNAME.delete(0, END); txtEID.delete(0, END)
        txtPHO.delete(0, END); txtBAL.delete(0, END)

    def insert():
        acct = txtACCT.get(); name = txtNAME.get(); eid = txtEID.get()
        pho = txtPHO.get(); bal = txtBAL.get()

```

```

try:
    myconn = get_connection()
    mycur = myconn.cursor()
    mycur.execute("INSERT INTO Bank VALUES (%s,%s,%s,%s,%s)", (acct, name
    myconn.commit()
    messagebox.showinfo("Insert", "Record Inserted")
    root.destroy(); insert_record()
except Exception as e:
    messagebox.showerror("Error", str(e))

root = Tk(); root.geometry("400x350"); root.title("Insert Record"); root.conf
Label(root, text="Account No:", bg="#ADD8E6").place(x=30, y=30)
txtACCT = Entry(root); txtACCT.place(x=150, y=30)
Label(root, text="Name:", bg="#ADD8E6").place(x=30, y=70)
txtNAME = Entry(root); txtNAME.place(x=150, y=70)
Label(root, text="EID:", bg="#ADD8E6").place(x=30, y=110)
txtEID = Entry(root); txtEID.place(x=150, y=110)
Label(root, text="Phone:", bg="#ADD8E6").place(x=30, y=150)
txtPHO = Entry(root); txtPHO.place(x=150, y=150)
Label(root, text="Balance:", bg="#ADD8E6").place(x=30, y=190)
txtBAL = Entry(root); txtBAL.place(x=150, y=190)
btnInsert = Button(root, text="Insert", command=insert, bg="#4682B4", fg="whi
btnInsert.place(x=60, y=250, width=80); btnInsert.bind("<Enter>", on_enter);
btnClear = Button(root, text="Clear", command=clear, bg="#4682B4", fg="white"
btnClear.place(x=180, y=250, width=80); btnClear.bind("<Enter>", on_enter); b
root.mainloop()

def display_records():
    try:
        myconn = get_connection()
        mycur = myconn.cursor()
        mycur.execute("""
            SELECT B.ACCTNO, B.NAME, B.EID, B.PHO, B.BAL,
                IFNULL(L.TOTAL_PAYABLE, 'NO') AS LOAN_AMOUNT
            FROM Bank B LEFT JOIN Loan L ON B.ACCTNO = L.ACCTNO
            ORDER BY B.ACCTNO
        """)
        data = mycur.fetchall()
        root = Tk(); root.geometry("800x400"); root.title("Bank Records"); root.c
        tree = ttk.Treeview(root, columns=(1, 2, 3, 4, 5, 6), show="headings", he
        tree.pack()
        tree.heading(1, text="ACCTNO"); tree.heading(2, text="NAME"); tree.headin
        tree.heading(4, text="PHONE"); tree.heading(5, text="BALANCE"); tree.head
        for d in data:
            tree.insert("", END, values=d)
    except Exception as e:

```

```

        messagebox.showerror("Error", str(e))

# ----- DEPOSIT -----
def deposit_money():
    def deposit():
        acct = txtACCT.get(); amt = Decimal(txtAMT.get())
        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute("SELECT BAL FROM Bank WHERE ACCTNO=%s", (acct,))
            bal = mycur.fetchone()[0]
            newbal = bal + amt
            mycur.execute("UPDATE Bank SET BAL=%s WHERE ACCTNO=%s", (newbal, acct))
            myconn.commit(); messagebox.showinfo("Deposit", f"Deposited {amt}, Ne
        except Exception as e:
            messagebox.showerror("Error", str(e))

    root = Tk(); root.geometry("300x200"); root.title("Deposit"); root.config(bg=
    Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
    txtACCT = Entry(root); txtACCT.place(x=120, y=30)
    Label(root, text="Amount:", bg="#ADD8E6").place(x=20, y=70)
    txtAMT = Entry(root); txtAMT.place(x=120, y=70)
    btnDep = Button(root, text="Deposit", command=deposit, bg="#4682B4", fg="whit
    btnDep.place(x=100, y=120, width=100); btnDep.bind("<Enter>", on_enter); btnD
    root.mainloop()

# ----- WITHDRAW -----
def withdraw_money():
    def withdraw():
        acct = txtACCT.get(); amt = Decimal(txtAMT.get())
        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute("SELECT BAL FROM Bank WHERE ACCTNO=%s", (acct,))
            bal = mycur.fetchone()[0]
            if bal >= amt:
                newbal = bal - amt
                mycur.execute("UPDATE Bank SET BAL=%s WHERE ACCTNO=%s", (newbal,
                myconn.commit(); messagebox.showinfo("Withdraw", f"Withdrew {amt}
            else:
                messagebox.showerror("Withdraw", "Insufficient Balance")
        except Exception as e:
            messagebox.showerror("Error", str(e))

    root = Tk(); root.geometry("300x200"); root.title("Withdraw"); root.config(bg

```

```

Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
txtACCT = Entry(root); txtACCT.place(x=120, y=30)
Label(root, text="Amount:", bg="#ADD8E6").place(x=20, y=70)
txtAMT = Entry(root); txtAMT.place(x=120, y=70)
btnWdr = Button(root, text="Withdraw", command=withdraw, bg="#4682B4", fg="wh
btnWdr.place(x=100, y=120, width=100); btnWdr.bind("<Enter>", on_enter); btnW
root.mainloop()

# ----- TAKE LOAN -----
def take_loan():
    def submit_loan():
        acct = txtACCT.get()
        principal = Decimal(txtPRINCIPAL.get())
        rate = Decimal(txtRATE.get())
        period = int(txtPERIOD.get())
        try:
            myconn = get_connection()
            mycur = myconn.cursor()

            mycur.execute("SELECT * FROM Loan WHERE ACCTNO=%s", (acct,))
            if mycur.fetchone():
                messagebox.showerror("Loan", "Loan already exists for this accoun
                return

            mycur.execute("SELECT * FROM Bank WHERE ACCTNO=%s", (acct,))
            if not mycur.fetchone():
                messagebox.showerror("Loan", "Account not found")
                return

            total = principal + (principal * rate * period) / 100
            mycur.execute("INSERT INTO Loan VALUES (%s,%s,%s,%s,%s)",
                           (acct, principal, rate, period, total))
            myconn.commit()
            messagebox.showinfo("Loan", "Loan successfully taken!\nTotal Payable:
            root.destroy()
        except Exception as e:
            messagebox.showerror("Error", str(e))

root = Tk(); root.geometry("400x300"); root.title("Take Loan"); root.config(b
Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
txtACCT = Entry(root); txtACCT.place(x=180, y=30)
Label(root, text="Loan Principal Amount:", bg="#ADD8E6").place(x=20, y=70)
txtPRINCIPAL = Entry(root); txtPRINCIPAL.place(x=180, y=70)
Label(root, text="Interest Rate (%):", bg="#ADD8E6").place(x=20, y=110)
txtRATE = Entry(root); txtRATE.place(x=180, y=110)
Label(root, text="Period (Months):", bg="#ADD8E6").place(x=20, y=150)

```

```

txtPERIOD = Entry(root); txtPERIOD.place(x=180, y=150)
Button(root, text="Take Loan", command=submit_loan, bg="#4682B4", fg="white")
root.mainloop()

def update_record():
    def update():
        acct = txtACCT.get()
        bal = txtBAL.get()
        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute("UPDATE Bank SET BAL=%s WHERE ACCTNO=%s", (bal, acct))
            myconn.commit()
            messagebox.showinfo("Update", "Balance Updated Successfully")
        except Exception as e:
            messagebox.showerror("Error", str(e))

    root = Tk()
    root.geometry("300x200")
    root.title("Update Balance")
    root.config(bg="#ADD8E6")

    Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
    txtACCT = Entry(root)
    txtACCT.place(x=120, y=30)

    Label(root, text="New Balance:", bg="#ADD8E6").place(x=20, y=70)
    txtBAL = Entry(root)
    txtBAL.place(x=120, y=70)

    btnUpdate = Button(root, text="Update", command=update, bg="#4682B4", fg="whi
    btnUpdate.place(x=60, y=120, width=80)
    btnUpdate.bind("<Enter>", on_enter)
    btnUpdate.bind("<Leave>", on_leave)

    root.mainloop()

def delete_record():
    def delete():
        acct = txtACCT.get()
        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute("DELETE FROM Loan WHERE ACCTNO=%s", (acct,))
            mycur.execute("DELETE FROM Bank WHERE ACCTNO=%s", (acct,))

```

```

        myconn.commit()
        messagebox.showinfo("Delete", "Record Deleted Successfully")
    except Exception as e:
        messagebox.showerror("Error", str(e))

root = Tk()
root.geometry("300x150")
root.title("Delete Record")
root.config(bg="#ADD8E6")

Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
txtACCT = Entry(root)
txtACCT.place(x=120, y=30)

btnDelete = Button(root, text="Delete", command=delete, bg="#4682B4", fg="whi
btnDelete.place(x=60, y=70, width=80)
btnDelete.bind("<Enter>", on_enter)
btnDelete.bind("<Leave>", on_leave)

root.mainloop()

# ----- LOAN REPAYMENT -----
def repay_loan():
    def show_amount():
        acct = txtACCT.get()
        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute("SELECT TOTAL_PAYABLE FROM Loan WHERE ACCTNO=%s", (acct
            res = mycur.fetchone()
            if res:
                txtAMOUNT.config(state=NORMAL)
                txtAMOUNT.delete(0, END)
                txtAMOUNT.insert(0, str(res[0]))
                txtAMOUNT.config(state=DISABLED)
            else:
                messagebox.showerror("Error", "No loan found for this account")
        except Exception as e:
            messagebox.showerror("Error", str(e))

def pay_loan():
    acct = txtACCT.get()
    try:
        myconn = get_connection()
        mycur = myconn.cursor()
        mycur.execute("DELETE FROM Loan WHERE ACCTNO=%s", (acct,))

```



```

        myconn.commit()
        messagebox.showinfo("Repayment", "Loan repaid successfully")
        root.destroy()
    except Exception as e:
        messagebox.showerror("Error", str(e))

root = Tk(); root.geometry("350x200"); root.title("Loan Repayment"); root.con
Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=30)
txtACCT = Entry(root); txtACCT.place(x=140, y=30)
Button(root, text="Show Amount", command=show_amount, bg="#4682B4", fg="white
Label(root, text="Amount to Pay:", bg="#ADD8E6").place(x=20, y=110)
txtAMOUNT = Entry(root, state=DISABLED)
txtAMOUNT.place(x=140, y=110)
Button(root, text="Pay Loan", command=pay_loan, bg="#4682B4", fg="white").pla
root.mainloop()

# ----- SEARCH RECORD -----
def search_record():
    def search():
        acct = txtACCT.get().strip()
        name = txtNAME.get().strip()
        eid = txtEID.get().strip()

        query = """
        SELECT B.ACCTNO, B.NAME, B.EID, B.PHO, B.BAL,
                IFNULL(L.TOTAL_PAYABLE, 'NO') AS LOAN
        FROM Bank B LEFT JOIN Loan L ON B.ACCTNO = L.ACCTNO
        WHERE 1=1
        """
        params = []
        if acct:
            query += " AND B.ACCTNO=%s"
            params.append(acct)
        if name:
            query += " AND B.NAME LIKE %s"
            params.append('%' + name + '%')
        if eid:
            query += " AND B.EID=%s"
            params.append(eid)

        try:
            myconn = get_connection()
            mycur = myconn.cursor()
            mycur.execute(query, params)
            result = mycur.fetchall()

```

```

        for item in tree.get_children():
            tree.delete(item)

        for r in result:
            tree.insert("", END, values=r)
    except Exception as e:
        messagebox.showerror("Error", str(e))

root = Tk()
root.geometry("800x400")
root.title("Search Record")
root.config(bg="#ADD8E6")

Label(root, text="Account No:", bg="#ADD8E6").place(x=20, y=20)
txtACCT = Entry(root)
txtACCT.place(x=120, y=20)

Label(root, text="Name:", bg="#ADD8E6").place(x=250, y=20)
txtNAME = Entry(root)
txtNAME.place(x=300, y=20)

Label(root, text="EID:", bg="#ADD8E6").place(x=430, y=20)
txtEID = Entry(root)
txtEID.place(x=460, y=20)

btnSearch = Button(root, text="Search", command=search, bg="#4682B4", fg="whi
btnSearch.place(x=580, y=17, width=80)
btnSearch.bind("<Enter>", on_enter)
btnSearch.bind("<Leave>", on_leave)

tree = ttk.Treeview(root, columns=(1, 2, 3, 4, 5, 6), show="headings", height
tree.pack(pady=60)
tree.heading(1, text="ACCTNO")
tree.heading(2, text="NAME")
tree.heading(3, text="EID")
tree.heading(4, text="PHONE")
tree.heading(5, text="BALANCE")
tree.heading(6, text="LOAN")

root.mainloop()

# ----- MAIN MENU -----
def main_menu():
    root = Tk()
    root.geometry("400x800")
    root.title("Bank Menu")

```

```

root.config(bg="#ADD8E6")

buttons = [
    ("Insert", insert_record),
    ("Display", display_records),
    ("Deposit", deposit_money),
    ("Withdraw", withdraw_money),
    ("Take Loan", take_loan),
    ("Repay Loan", repay_loan),
    ("Search", search_record),
    ("Update", update_record),
    ("Delete", delete_record)
]

y = 50
for bname, bfunc in buttons:
    btn = Button(root, text=bname, command=bfunc, bg="#4682B4", fg="white", f
    btn.place(x=100, y=y, width=200, height=50)
    btn.bind("<Enter>", on_enter)
    btn.bind("<Leave>", on_leave)
    y += 70

root.mainloop()

# ----- MAIN -----
if __name__ == "__main__":
    init_db()
    rainbow_turtle()

```